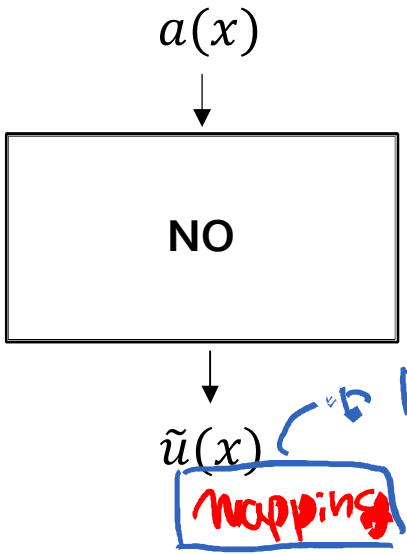
56262

Neural Operator 2

Neural Operator: Learning Maps Between Function Spaces With Applications to PDEs
(*Journal of Machine Learning Research, 2021)*

Neural Operator



- **Key Advantages**
  - Works on any resolution / mesh
  - Same parameters across discretizations
  - Consistent in function space
  - Naturally suited for PDE problems

$$G : a(x) \rightarrow u(x)$$

- Maps input function to solution function

Formulation

$$G_\theta : a(x) \to u(x)$$

- **Architecture Overview**

1. Lifting $a(x) \to v_0(x)$

   enc

   Data → Feature

   - A pointwise transformation
   - Maps input to a **higher-dimensional feature space**

2. Iterative Kernel Integration $v_t \to v_{t+1}$

   physic

$$v_{t+1}(x) = \sigma\left(W v_t(x) + \int \kappa(x, y) v_t(y) dy + b(x)\right)$$

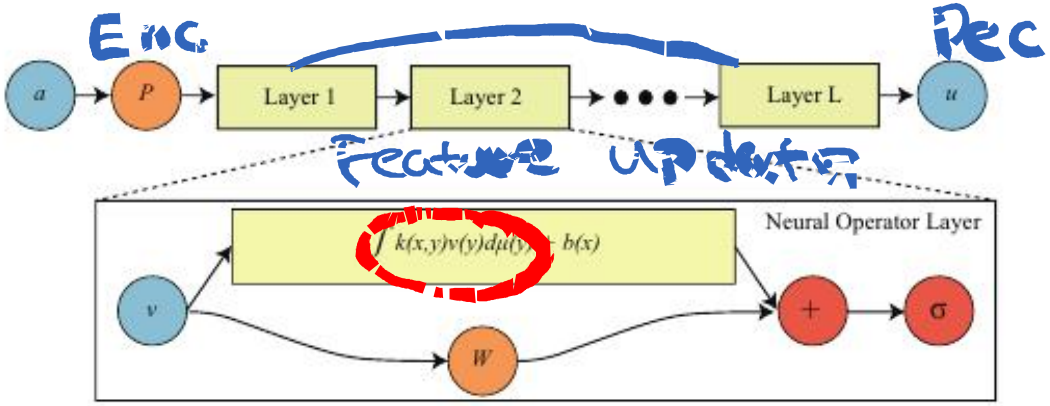
3. Projection $v_T(x) \to u(x)$

   dec

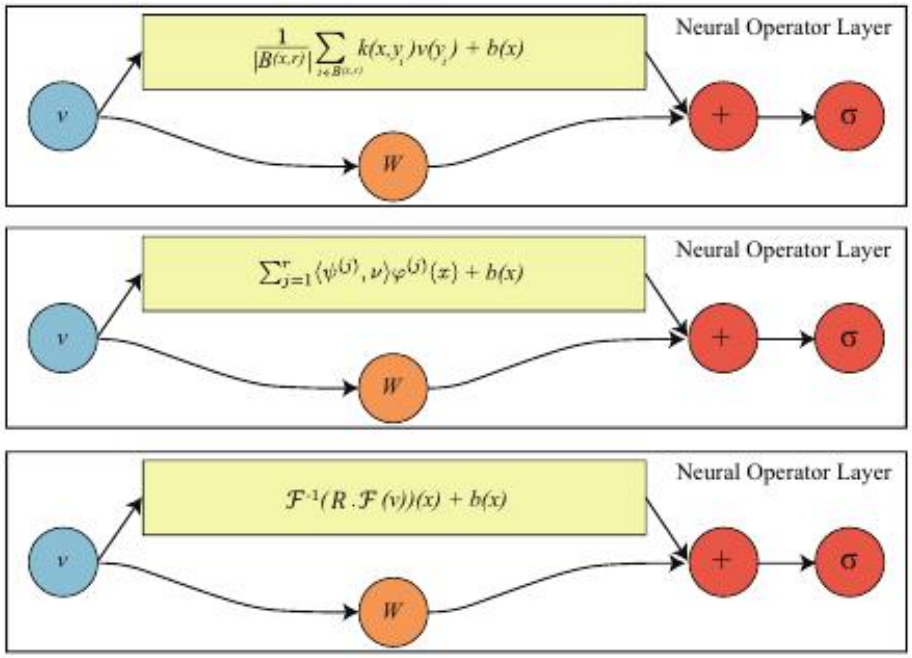
   - maps features back to output function

   Feature → data

Implementation Variants (Kernel)



Instantiations of neural operator layer



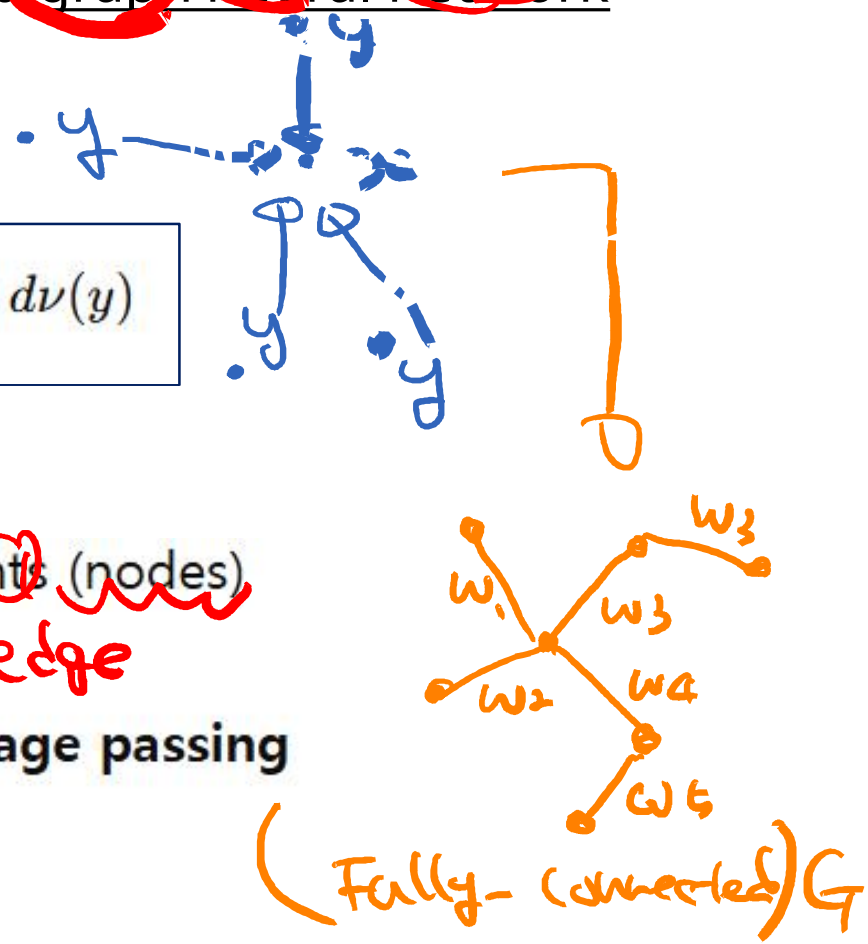
- Graph Neural Operator (GNO)
- Low-Rank Neural Operator (LNO)
- Multipole Neural Operator (MGNO)
- Fourier Neural Operator (FNO)

Graph Neural Operator 1/3

- **Core Idea:** Integral operator → Graph message passing
  - Graph Neural Operator (GNO) is a method to approximate a continuous integral operator using a graph neural network (GNN) framework.
  - The original operator is

$$u(x) = \int_D \kappa(x, y)\, v(y)\, d\nu(y)$$

  - GNO approximates this by:
    - Discretizing the domain into points (nodes)
    - Treating them as a **graph**
    - Replacing the integral with **message passing**



Graph Neural Operator 2/3

- **GNO as a Graph Neural Network**
  - **Nodes**: spatial points x_i
  - **Edges**: connections (local or global)
  - **Message passing**: $\kappa(x_i, x_j)\, v(x_j)$

The kernel function $\kappa(x, y)$ is **learned as an edge function**

- Integration → aggregation over neighbors
- Kernel learning → edge weight learning

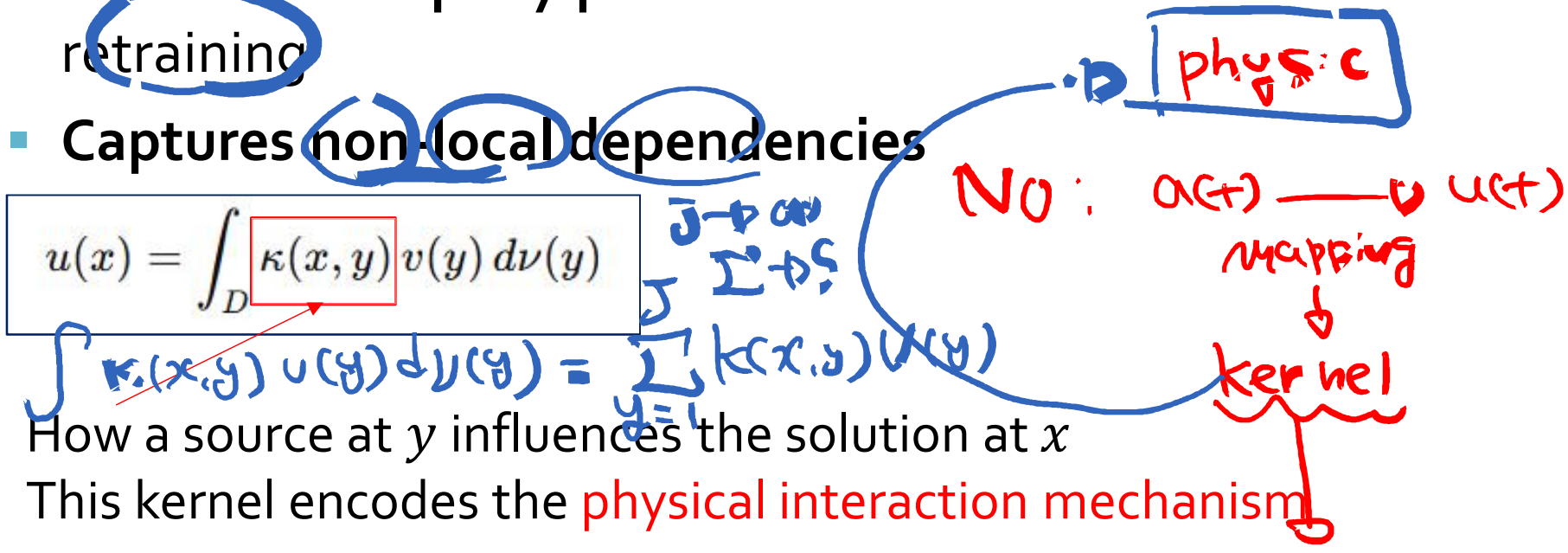
Graph Neural Operator 3/3

- **Key Advantages of GNO**
  - **Discretization-invariant**: works on different grids
  - **Handles new query points**: can add nodes without retraining
  - **Captures non-local dependencies**

$$u(x) = \int_D \kappa(x, y)\, v(y)\, d\nu(y)$$

How a source at y influences the solution at x

This kernel encodes the physical interaction mechanism



GNO learns diffusion, waves, and fluid dynamics by shaping the kernel through optimization so that it reproduces the correct spatial interaction patterns required to minimize prediction error.

Low-rank Neural Operator 1/3

- **Core Idea:** Integral operator → Low-rank decomposition
  - Low-rank Neural Operator (LNO) approximates the kernel of a neural operator using a low-rank decomposition, which significantly reduces computational cost while preserving expressive power.
  - The original operator is

$$u(x) = \int_D \kappa(x, y)\, v(y)\, d\nu(y)$$

- $\kappa(x, y)$: kernel function (learned)
- This is expensive: **O(N²)** interactions

Low-rank Neural Operator 2/3

- **Low-rank decomposition**
  - LNO assumes the kernel can be approximated as:

$$\kappa(x, y) \approx \sum_{k=1}^{r} \phi_k(x)\, \psi_k(y)$$

output basis input basis

- r: rank (small)
- $\phi_k(x)$: output basis functions
- $\psi_k(y)$: input basis functions

$\phi_k(x) \leftarrow \psi_k(y)$

  - Key Interpretation

1. **Project input** $v(y)$ onto basis $\psi_k(y)$
2. **Combine with output basis** $\phi_k(x)$

Low-rank Neural Operator 3/3

- **Computational Benefit**
  - The kernel doesn't need full N × N complexity
  - LNO learns a small set of global interaction patterns
    - Original: **O(N²)**
    - Low-rank: **O(Nr)**
- **Limitation**
  - Highly nonlocal interactions
  - Very complex kernels, because rank r is limited

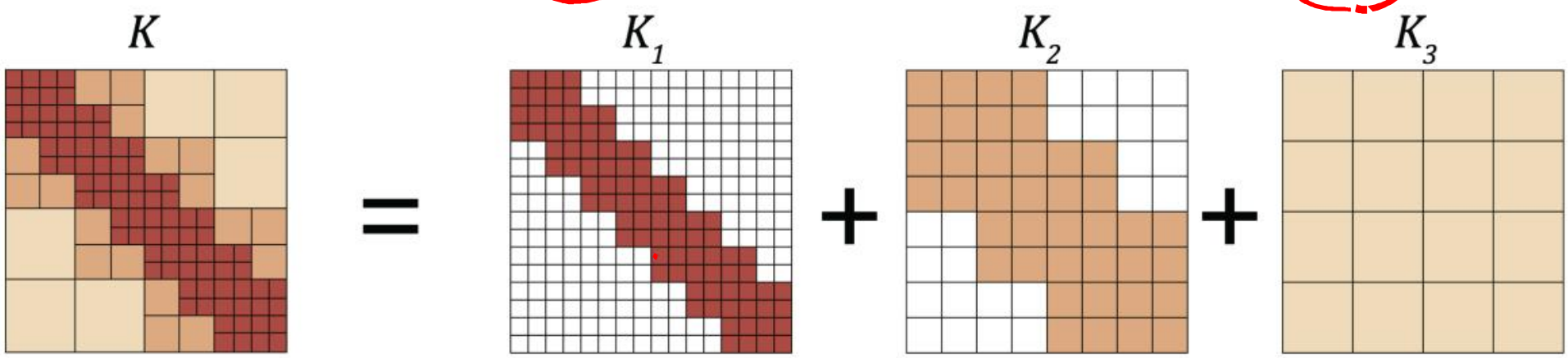
Multipole Neural Operator 1/3

- **Core Idea:** Integral operator → Hierarchical, multipole-style approximations
  - Multipole Neural Operator (MNO) is designed to efficiently model long-range interactions in neural operators by using a hierarchical, multipole-style decomposition

- **Key Insight**
  - Instead of computing interactions directly between every pair (x, y), MNO groups points hierarchically and approximates interactions between groups

Multipole Neural Operator 2/3

- **Hierarchical Structure**
  - MNO builds a **tree structure** (like quadtree / octree):
    - Fine level → detailed local interactions
    - Coarse level → summarized global interactions



Hierarchical matrix decomposition

Multipole Neural Operator 3/3

- **Computational Benefit**
  - Standard operator: **O(N²)**
  - MNO: **O(N log N)** or even **O(N)** (depending on implementation)
- **Strengths**
  - Efficient for **large-scale problems**
  - Captures **both local and global interactions**
  - Works well for **physics simulations**
- **Limitations**
  - More complex architecture
  - Requires careful design of hierarchy
  - May be harder to train than simpler operators

Fourier Neural Operator 1/3

- **Core Idea:** Integral operator → Fourier space
  - Fourier Neural Operator (FNO) learns mappings between functions by performing global convolution in Fourier (frequency) space.
  - The original operator is

$$u(x) = \int_D \kappa(x, y)\, v(y)\, d\nu(y)$$

  - FNO transforms the problem into:

$$\hat{u}(k) = R_\theta(k)\, \hat{v}(k)$$

    - $\hat{v}(k)$: Fourier transform of input
    - $R_\theta(k)$: learned transformation in frequency space

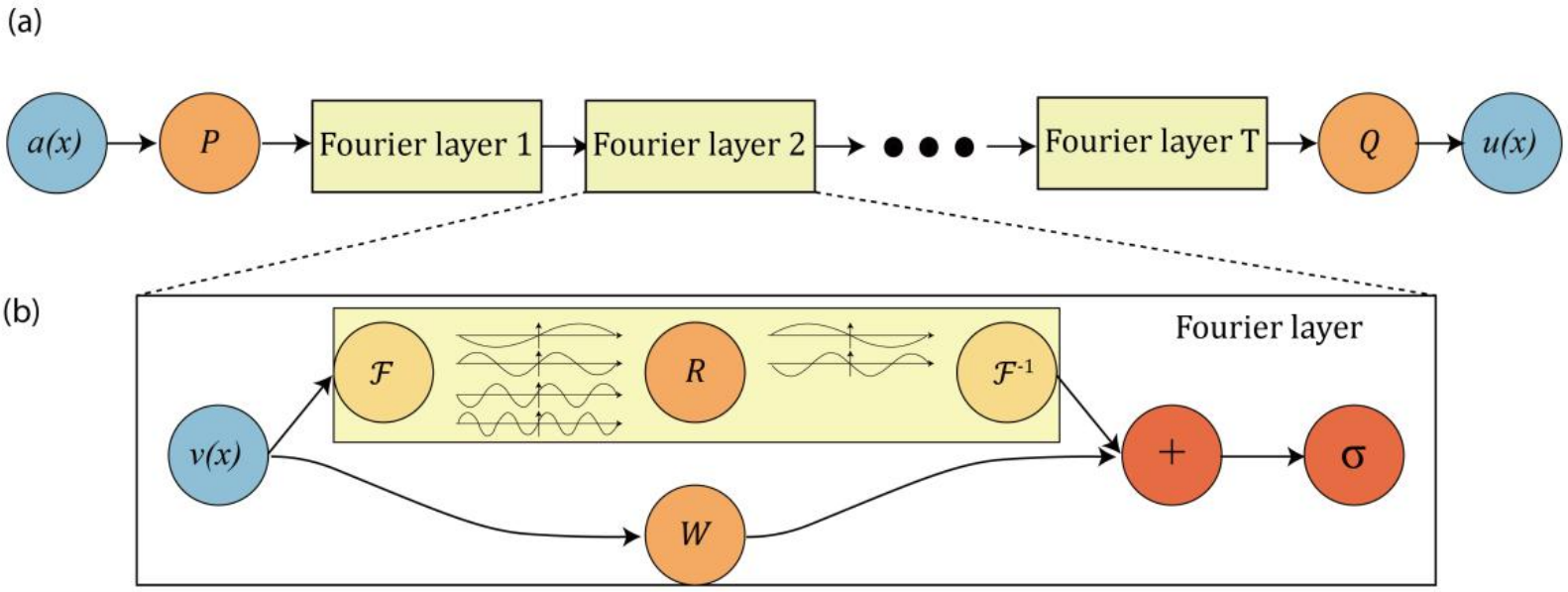
Fourier Neural Operator 2/3

- **Key Procedure**
  - **Step 1:** Fourier Transform $v(x) \rightarrow \hat{v}(k)$
  - **Step 2:** Spectral Transformation $\hat{v}(k) \rightarrow R_\theta(k)\,\hat{v}(k)$
    - Only **low-frequency modes** are typically kept
    - High-frequency modes are truncated

    Low-frequency truncation
  - **Step 3:** Inverse Fourier Transform $\hat{u}(k) \rightarrow u(x)$
  - **Step 4**: Add Local Linear Layer $u(x) = \mathcal{F}^{-1}(R_\theta \hat{v}) + W v(x)$
    - **global interaction (Fourier)**
    - **local correction (linear layer)**

Fourier Neural Operator 3/3

- **Strengths**
  - **Computationally efficient:** Fourier transform converts convolution into multiplication
  - **Physical Interpretation:** FNO learns how different frequency modes interact
  - **Works well for:** CFD, weather prediction, diffusion systems



Summary

- **GNO**: Subsample J' points from the J-point discretization and compute the truncated integral

Graph

$$u(x) = \int_{B(x,r)} \kappa(x, y)\, v(y)\, \mathrm{d}y \tag{28}$$

- **LNO**: Decompose the kernel function tensor product form and compute

Low-rank

$$u(x) = \sum_{j=1}^{r} \langle \psi^{(j)}, v \rangle \varphi^{(j)}(x) \tag{29}$$

- **MGNO**: Compute a multi-scale decomposition of the kernel

Multi-scale

$$K = K_{1,1} + K_{1,2}K_{2,2}K_{2,1} + K_{1,2}K_{2,3}K_{3,3}K_{3,2}K_{2,1} + \cdots$$
$$u(x) = (Kv)(x) \tag{30}$$

- **FNO**: Parameterize the kernel in the Fourier domain and compute the using the FFT

Fourier

$$u(x) = \mathcal{F}^{-1}(R_\phi \cdot \mathcal{F}(v))(x) \tag{31}$$

Comparisons

Aspect	GNO	LNO	MNO	FNO
Core idea	Full kernel learning (graph-based)	Low-rank factorization	Hierarchical multipole	Fourier transform
Interaction	All-to-all (explicit)	Global (compressed)	Local + global (hierarchical)	Global (frequency)
Structure	Graph / message passing	Basis decomposition	Tree / multi-scale	Spectral
Complexity	$O(N^2)$	O(Nr)	O(N log N)	O(N log N)
Strength	Most general, flexible, works on irregular meshes	Efficient for low-rank systems	Efficient long-range interactions	Strong for smooth PDEs
Weakness	Poor scalability (expensive)	Limited expressiveness for complex physics	Complex design	Struggles with non-periodic / irregular domains